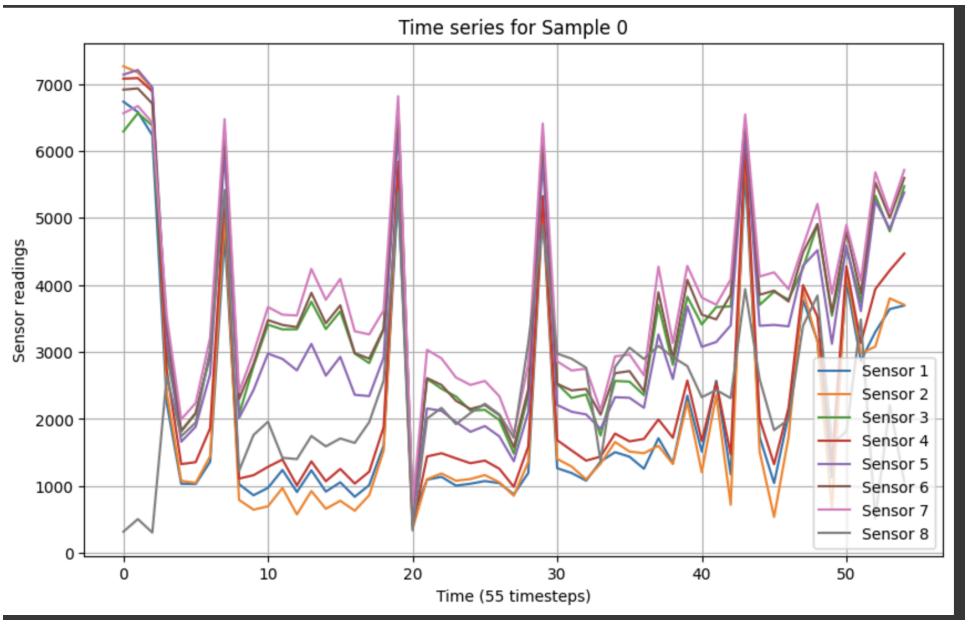
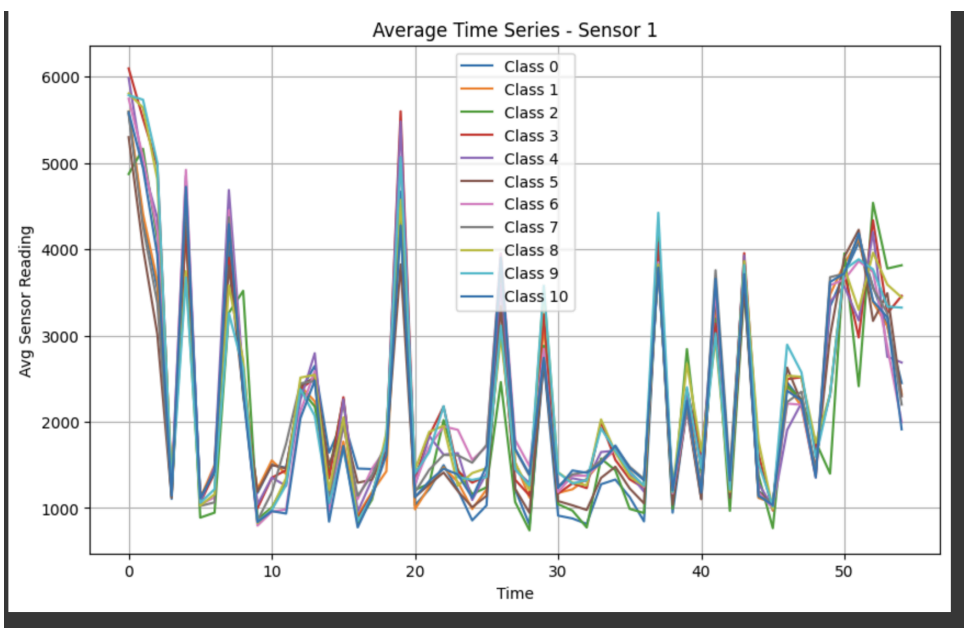
Deep Learning Project - Time-Series Sensor Data

1. Plotting and Visualisation:

Plots:



Graph 1



Graph 2

Insights:

Graph 1 plots Sample 0's time series of sensor readings with a unique color for one of the 8 sensors over 55 timesteps. Sensors exhibit both synchronous and superpositioning peaks at somewhat regular intervals, indicating correlated sensor behavior over time. This guided the use of models that preserve temporal and local patterns, i.e., 1D CNNs or 2D CNNs along time and sensor directions.

Graph 2 plots the mean time series of Sensor 1 for all classes (0–10). Although it presents oscillations, the curves are not separable enough and are not periodic.
This demonstrated that between-class tendencies are small and will likely be neglected by averaging, implies using deep models which are capable of learning non-linear class-related features, i.e., deeper layered CNNs or RNNs.

2. Architectures of Best Models

CNN Best Model Architecture:

```
CNN
_modules:
  dict
    conv:
      Sequential
        _modules:
          dict
            0:
              Conv1d
                weight ⟨32×8×3⟩
                bias ⟨32⟩
            1:
              BatchNorm1d
                weight ⟨32⟩
                bias ⟨32⟩
                running_mean ⟨32⟩
                running_var ⟨32⟩
                num_batches_tracked = 28500
            2:
              ReLU
            3:
              MaxPool1d
            4:
              Conv1d
                weight ⟨64×32×3⟩
                bias ⟨64⟩
            5:
              BatchNorm1d
                weight ⟨64⟩
                bias ⟨64⟩
                running_mean ⟨64⟩
                running_var ⟨64⟩
                num_batches_tracked = 28500
            6:
              ReLU
            7:
              MaxPool1d
            8:
              AdaptiveAvgPool1d
    classifier:
      Sequential
        _modules:
          dict
            0:
              Flatten
            1:
              Linear
                weight ⟨32×64⟩
                bias ⟨32⟩
            2:
              ReLU
            3:
              Linear
                weight ⟨11×32⟩
                bias ⟨11⟩
```

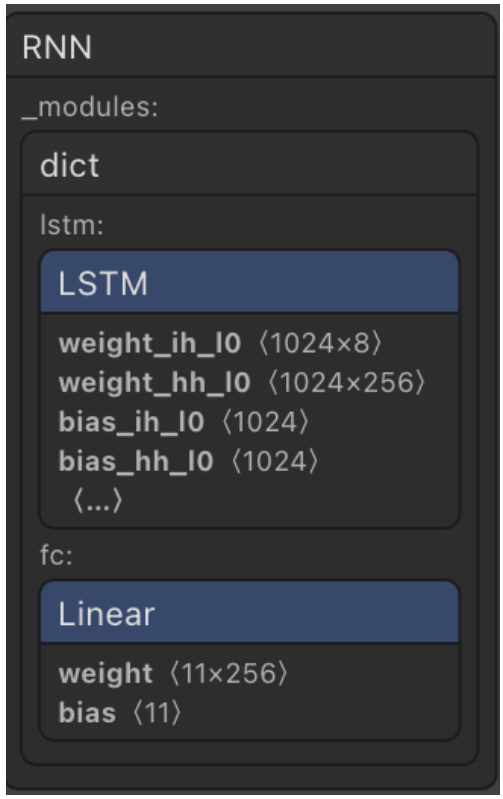
```python
class CNN(torch.nn.Module):
 def __init__(self, num_classes):
   super(CNN, self).__init__()

   self.conv = torch.nn.Sequential(
       torch.nn.Conv1d(8, 32, kernel_size=3, stride=1, padding=1),
       torch.nn.BatchNorm1d(32),
       torch.nn.ReLU(),
       torch.nn.MaxPool1d(kernel_size=2, stride=2),

       torch.nn.Conv1d(32, 64, kernel_size=3, stride=1, padding=1),
       torch.nn.BatchNorm1d(64),
       torch.nn.ReLU(),
       torch.nn.MaxPool1d(kernel_size=2, stride=2),

       torch.nn.AdaptiveAvgPool1d(1),
   )

   self.classifier = torch.nn.Sequential(
       torch.nn.Flatten(),
       torch.nn.Linear(64, 32),
       torch.nn.ReLU(),
       torch.nn.Linear(32, num_classes)
   )

 def forward(self, x):
   x = x.permute(0, 2, 1)  #to convert(batch, seq_len, features) ->
(batch, features, seq_len)
   x = self.conv(x)
   x = self.classifier(x)
   return x
```

Rationale:

In this architecture, 1D convolutions are employed to extract patterns within the input features, batch normalization, activation with ReLU, and max pooling to reduce dimensionality and

improve generalization, and the final layers make predictions based on the features. AdaptiveAvgPool gives output of fixed size regardless of the input length.

RNN Best Model Architecture:




```
class RNN(torch.nn.Module):
 def __init__(self, input_size, hidden_size, output_size):
   super(RNN, self).__init__()
   self.hidden_size = hidden_size
   self.lstm = torch.nn.LSTM(input_size, hidden_size, batch_first=True)
   self.fc = torch.nn.Linear(hidden_size, output_size)

 def forward(self, x):
   x, h = self.lstm(x)
   x = self.fc(x[:, -1, :])
   return x
```

Rationale:

The LSTM-based RNN learns to discover time step sequential dependencies via an evolving hidden state. This only utilizes the last hidden state to perform the classification, and this is suitable where the classification issue at hand is a sequence classification where the last state holds useful context of the entire sequence.

Autoencoder Best Model Architecture:

```
AutoencoderClassifier
_modules:
  dict
    encoder:
      Sequential
        _modules:
          dict
            0:
              Conv1d
                weight ⟨16×8×3⟩
                bias ⟨16⟩
            1:
              ReLU
            2:
              Conv1d
                weight ⟨32×16×3⟩
                bias ⟨32⟩
            3:
              ReLU
            4:
              Flatten
            5:
              Linear
                weight ⟨10×448⟩
                bias ⟨10⟩
    classifier:
      Sequential
        _modules:
          dict
            0:
              Linear
                weight ⟨32×10⟩
                bias ⟨32⟩
            1:
              BatchNorm1d
                weight ⟨32⟩
                bias ⟨32⟩
                running_mean ⟨32⟩
                running_var ⟨32⟩
                num_batches_tracked = 28500
            2:
              ReLU
            3:
              Dropout
            4:
              Linear
                weight ⟨11×32⟩
                bias ⟨11⟩
```

```python
class Autoencoder(torch.nn.Module):
    def __init__(self):
        super(Autoencoder, self).__init__()

        #Encoder
        self.encoder = torch.nn.Sequential(
            torch.nn.Conv1d(in_channels=8, out_channels=16, kernel_size=3,
stride=2, padding=1),
            torch.nn.ReLU(),
            torch.nn.Conv1d(16, 32, kernel_size=3, stride=2, padding=1),
            torch.nn.ReLU(),
            torch.nn.Flatten(),
            torch.nn.Linear(448, 10)
        )

        #Decoder
        self.decoder = torch.nn.Sequential(
            torch.nn.Linear(10, 32 * 14),
            torch.nn.Unflatten(1, (32, 14)),
            torch.nn.ConvTranspose1d(32, 16, kernel_size=3, stride=2,
padding=1, output_padding=1),
            torch.nn.ReLU(),
            torch.nn.ConvTranspose1d(16, 8, kernel_size=3, stride=2,
padding=1, output_padding=0),
        )

    def encode(self, x):
        x = x.permute(0, 2, 1) #[32,8,5]
        latent = self.encoder(x)  #[32,10]
        return latent

    def decode(self, latent):
        x = self.decoder(latent)  #[32,8,55]
        x = x.permute(0, 2, 1)  #[32,55,8]
        return x

    def forward(self, x):
        latent = self.encode(x)
        return self.decode(latent)
```

```python
class AutoencoderClassifier(torch.nn.Module):
    def __init__(self, autoencoder, num_classes=10):
        super(AutoencoderClassifier, self).__init__()
        self.encoder = autoencoder.encoder
        self.classifier = torch.nn.Sequential(
            torch.nn.Linear(10, 32),
            torch.nn.ReLU(),
            torch.nn.Linear(32, num_classes)
        )

    def forward(self, x):
        x = x.permute(0, 2, 1)
        latent = self.encoder(x)
        out = self.classifier(latent)
        return out
```

Rationale:
The architecture utilizes a pre-trained encoder of an autoencoder to recover useful compressed features of data and classify them with a feedforward network classifier. The autoencoder is written without MaxPooling to retain maximum feature representation - rather than only the significant ones. Initially the encoder- decoder architecture is trained and then the pretrained encoder is used along with the classifier to predict the classes based on learned latent representation.

3. Hyperparameters

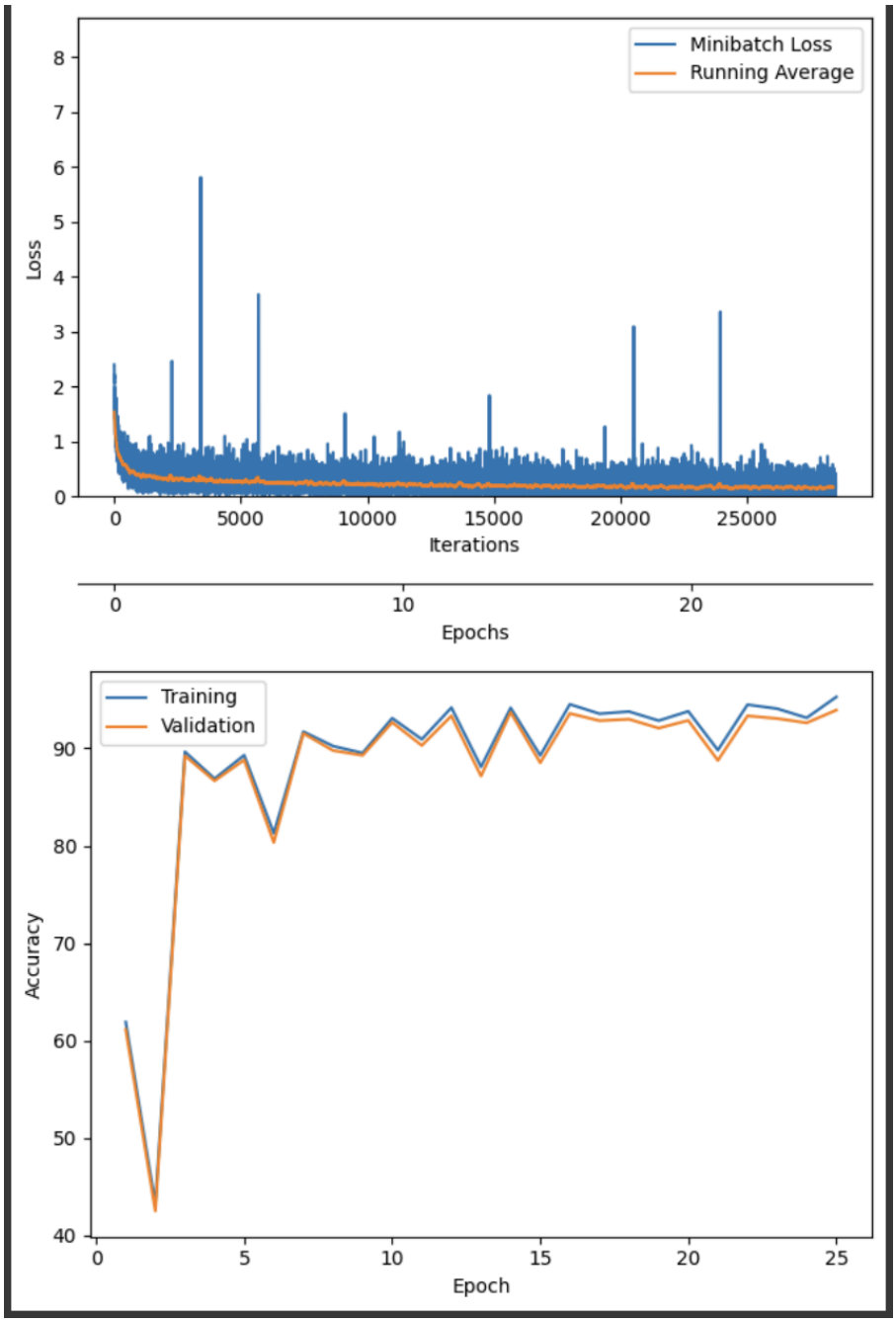
Note: The optimum set of parameters are highlighted in bold in the table.

<u>CNN:</u>

Step	Activation	Learning rate	Optimizer	Other Hyperparameters	Batch size	epochs	Train Accuracy	Validation Accuracy	Test Accuracy
1	ReLU	0.001	SGD		32	25	11.07	11.19	11.20
2	ReLU	0.0001	SGD		32	25	85.31	85.21	85.69
3	ReLU	0.0001	SGD with m =		64	25	74.90	74.65	75.40

			0.9						
4	ReLU	0.001	Adam		32	25	93.66	93.48	92.96
5	***ReLU***	***0.001***	***Adam***	***Preactivation batchnorm applied***	***32***	***25***	***94.86***	***94.02***	***93.96***

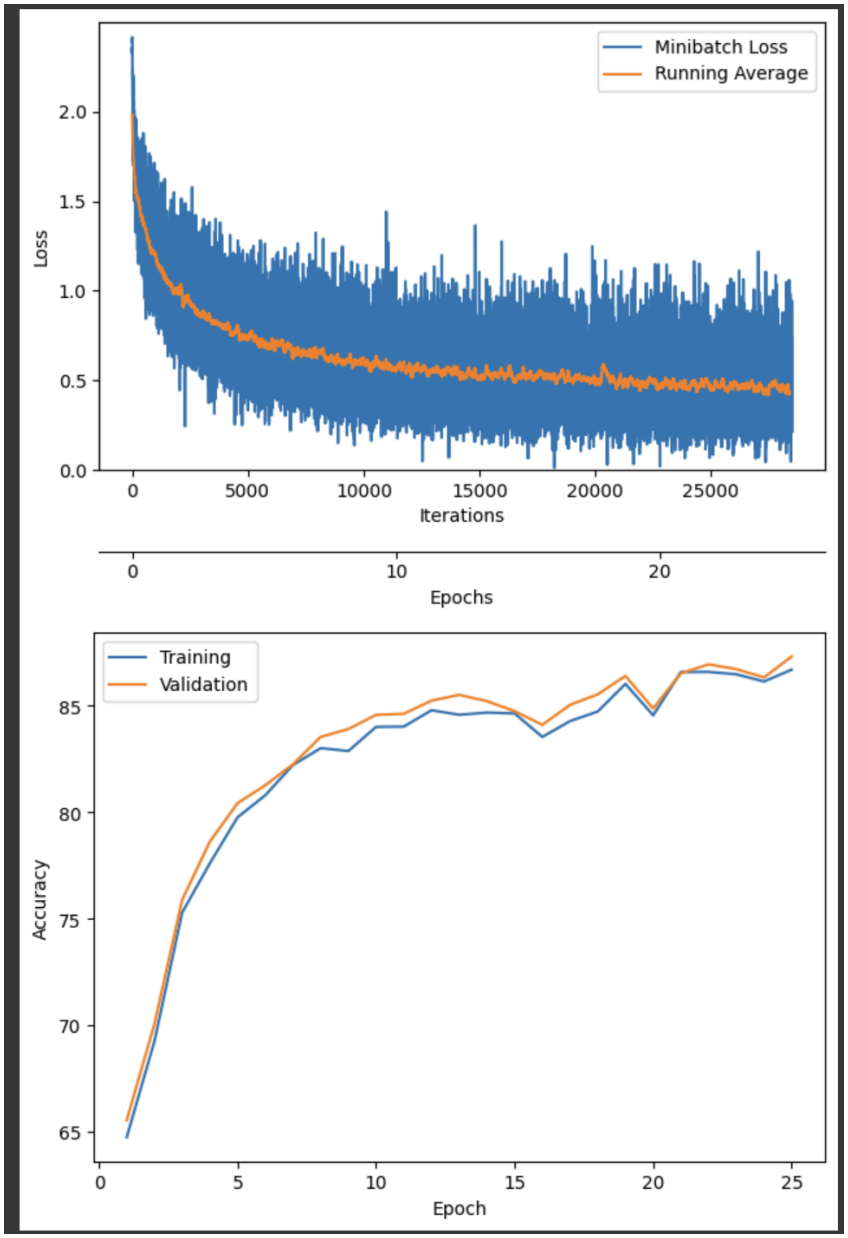
Optimum Hyperparameters - Loss curves and Accuracy Curves



RNN:

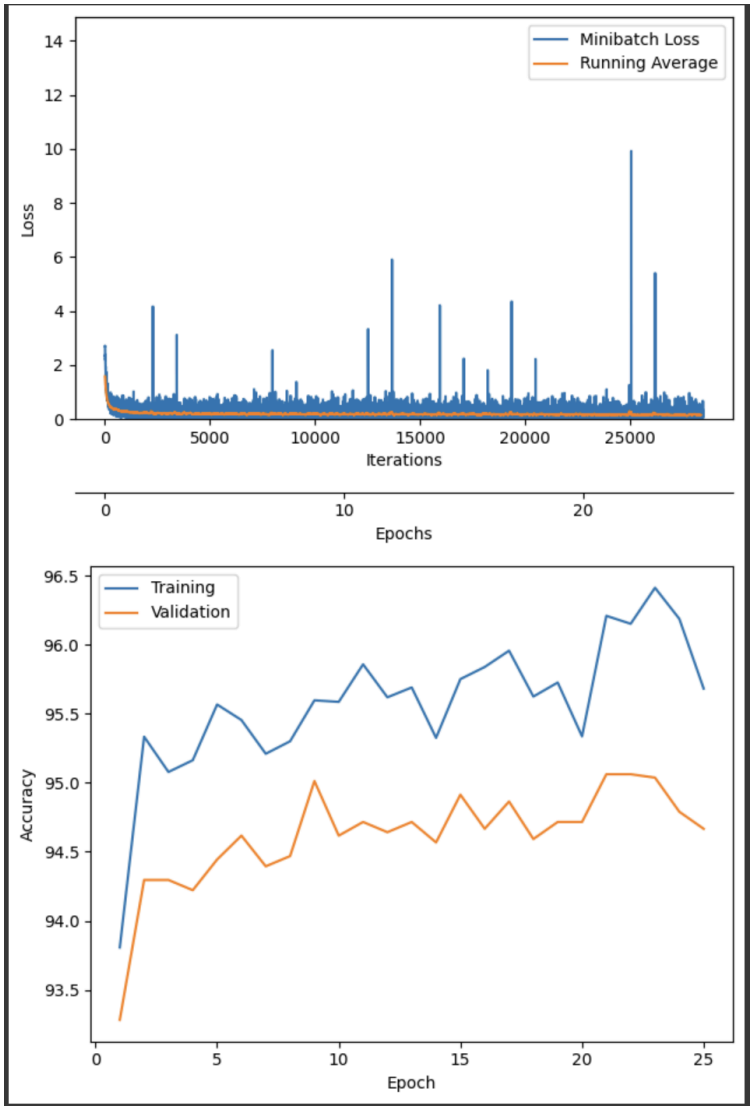
Step	Activation	Learning rate	Optimizer	Hidden size	Batch size	epochs	Train Accu racy	Valid ation Accu racy	Test Accu racy
1	Tanh and Sigmoid in LSTM	0.0001	adam	64	32	25	83.30	83.23	83.07
2	Tanh and Sigmoid in LSTM	0.0001	adam	128	32	25	85.06	85.58	84.64
3	***Tanh and Sigmoid in LSTM***	***0.0001***	***adam***	***256***	***32***	***25***	***86.69***	***87.31***	***86.24***
4	Tanh and Sigmoid in LSTM + ReLU For FC Layer	0.0001	adam	256	32	25	85.68	85.88	85.42

Optimum Hyperparameters - Loss and Accuracy Curves:


Minibatch Loss
Running Average
Loss
2.0
1.5
1.0
0.5
0.0
0
5000
10000
15000
20000
25000
Iterations
0
10
20
Epochs
Training
Validation
Accuracy
85
80
75
70
65
0
5
10
15
20
25
Epoch

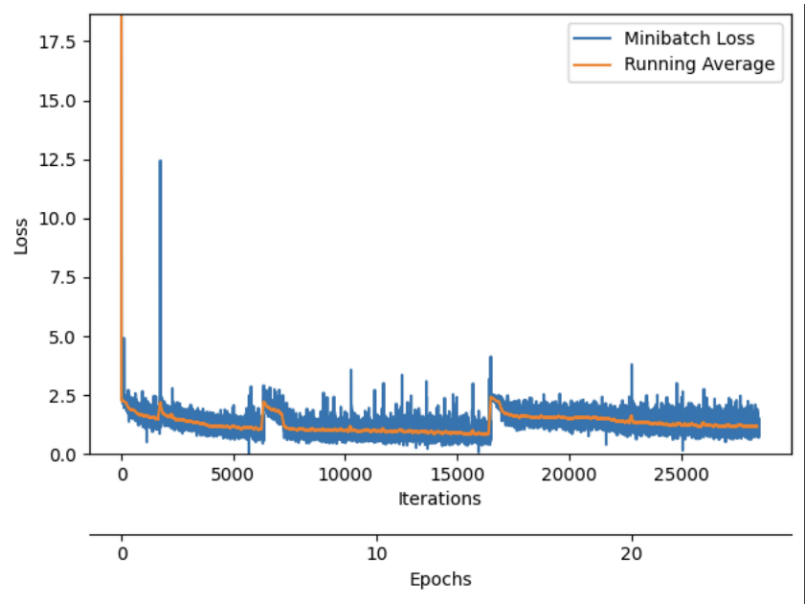
Autoencoder:

Step	Activation	Learning rate	Optimizer	Other Parameters	Batch size	epochs	Train Accuracy	Validation Accuracy	Test Accuracy
1	ReLU	0.001	Adam		32	25	92.53	92.02	92.00
2	relu	0.001	Adam	LRScheduler - ReduceLROnPlateau	32	25	93.99	93.41	93.42
3	Relu	0.001	Adam	Pre-activation BatchNorm + LRScheduler	32	24	96.36	94.99	94.80
4	***Relu***	***0.001***	***Adam***	***BatchNorm + Lr scheduler + Dropout prob = 0.3***	***32***	***25***	***95.68***	***94.67***	***94.84***

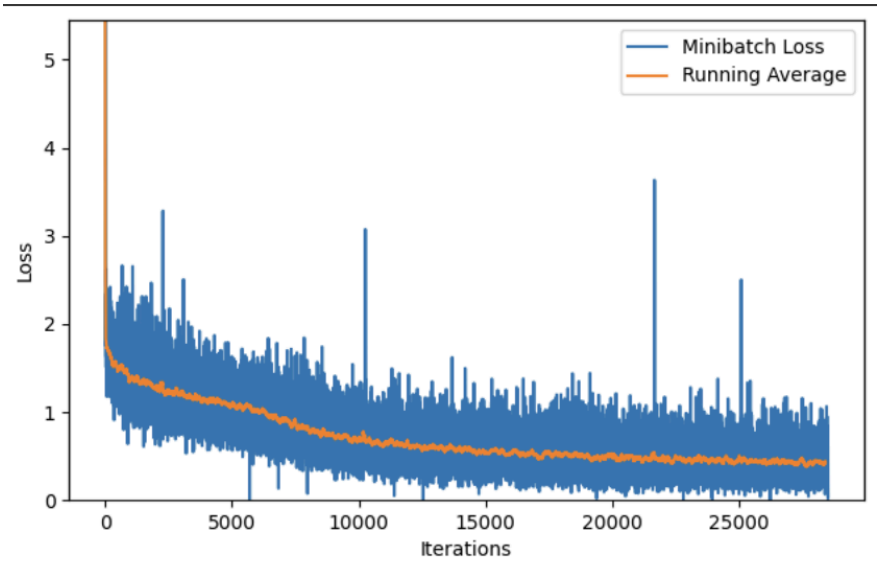


1. **Rationale - Observations: For the choice of Hyperparameters**

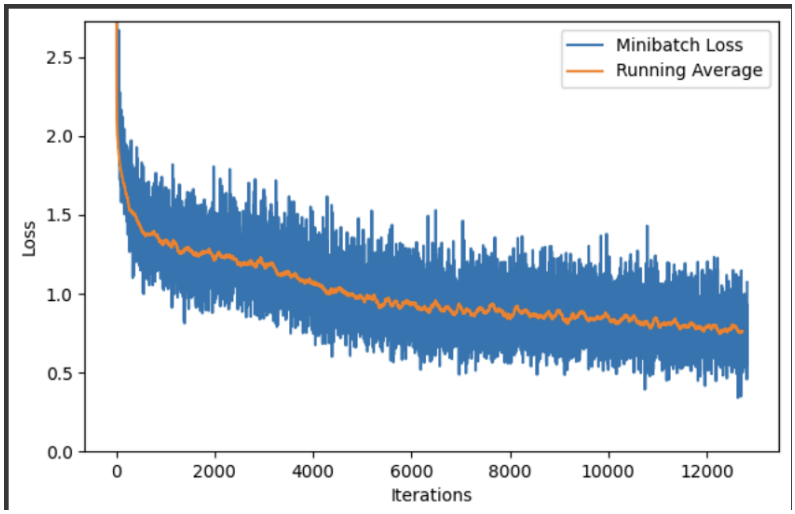
CNN:



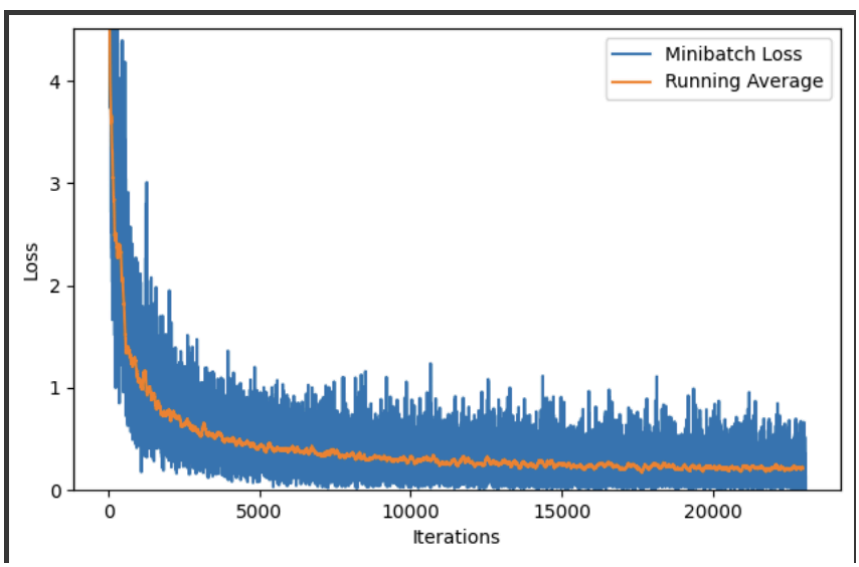
With learning rate 0.001 and SGD, the updates were noisy - with high fluctuations, the accuracy was very low, So decreased the learning rate even further to 0.0001.
It resulted in a steady decrease of loss and good accuracy.



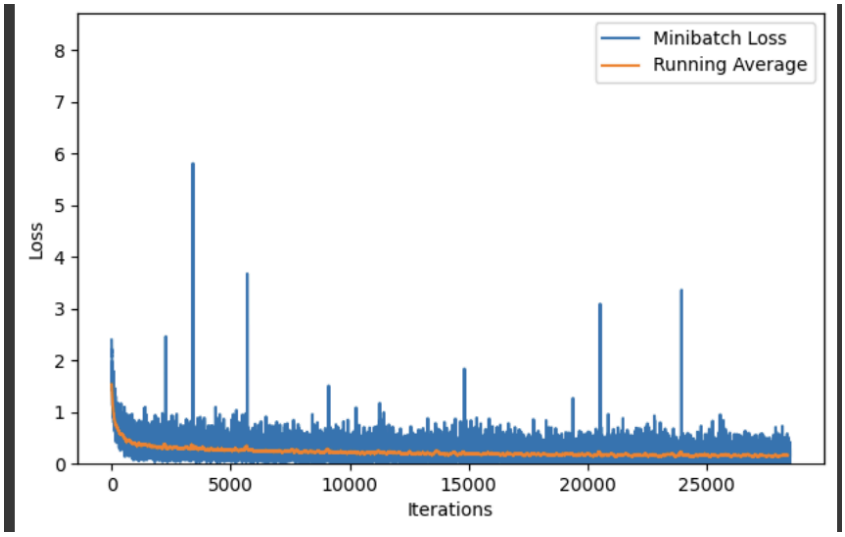
Tried adding the momentum to the learning rate for even more inclined (accelerate convergence towards the minimum) updates. It resulted in the following graph:



The loss value did not seem to converge effectively to the minimum, so tried with a different optimizer - adam. It resulted in a better loss convergence graph - the loss value was way lower than the previous one and the accuracy improved significantly.

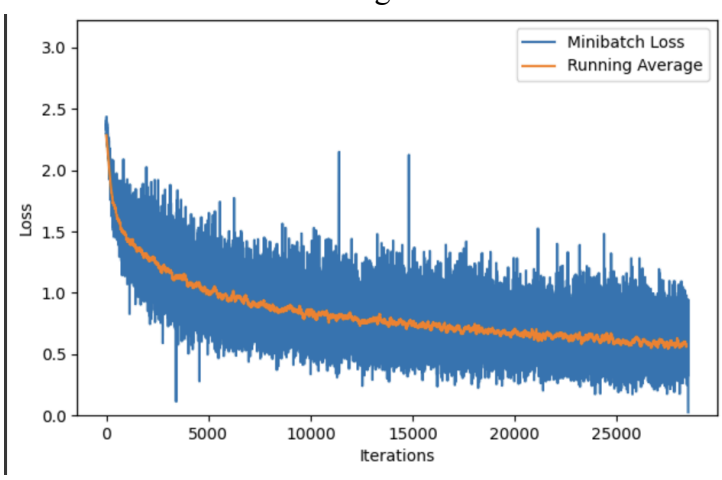


BatchNorm1d was added after each convolutional layer to maintain stable activations and improve generalization during training. This led to faster training process and helped faster convergence. This is the graph that was obtained after applying batchnorm:

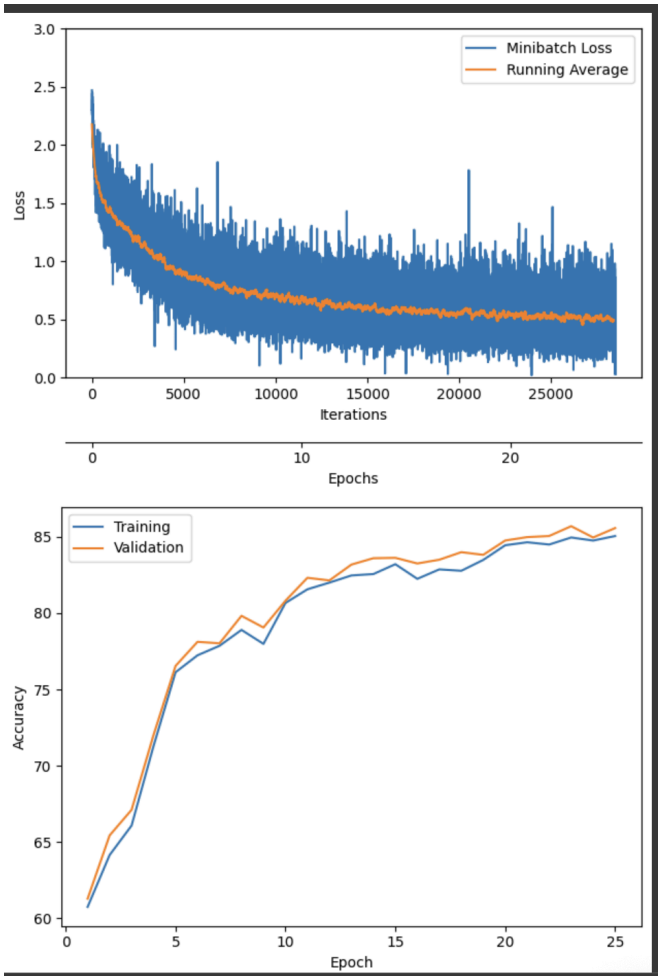


RNN:

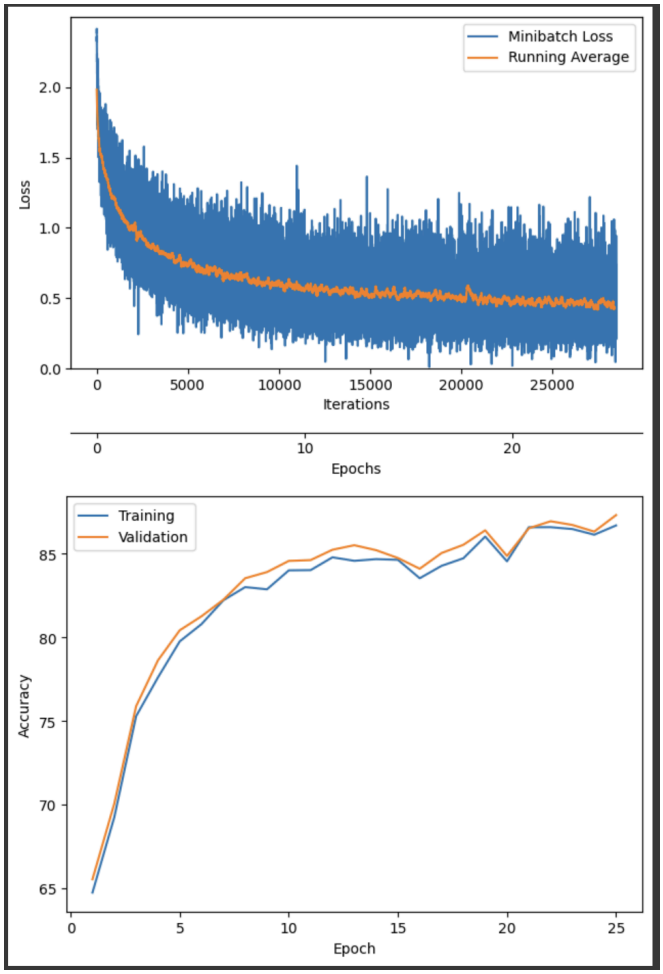
Initial loss curve with learning rate = 0.0001 and hidden size = 64



The loss did not reach the minimum though the learning rate was low, so tried to increase the model complexity so that the model can learn more - increased the hidden state to 128 from 64. This resulted in better accuracy and lower loss.



So, further increased the model's hidden size as it has resulted in a better accuracy is the previous increase.



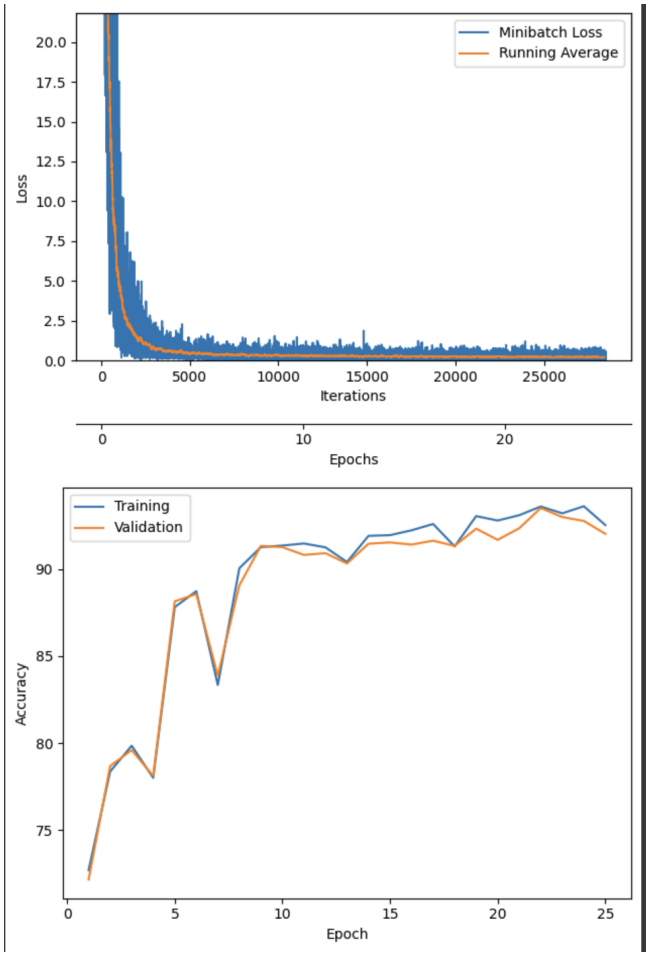
This resulted in even better accuracy and lower loss.

Then added ReLU activation for the fully connected layer - it did not add much to the accuracy - resulted in a slightly lesser accuracy - so retained the previous model.

Autoencoder:

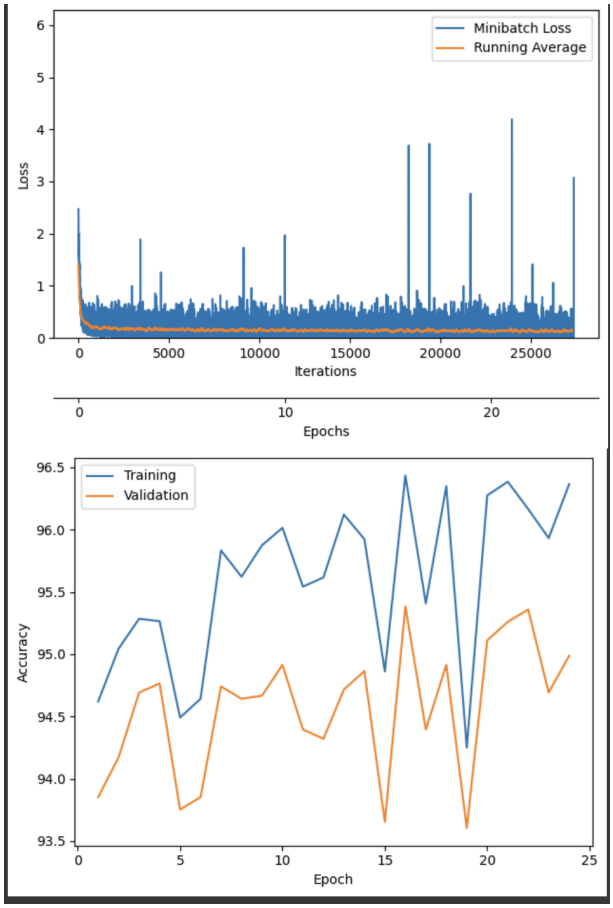
As the encoder part of the model was already pre trained using MSE Loss (the encoder-decoder part) the model scored high accuracy without much Hyperparameter tuning.

Initial graph:

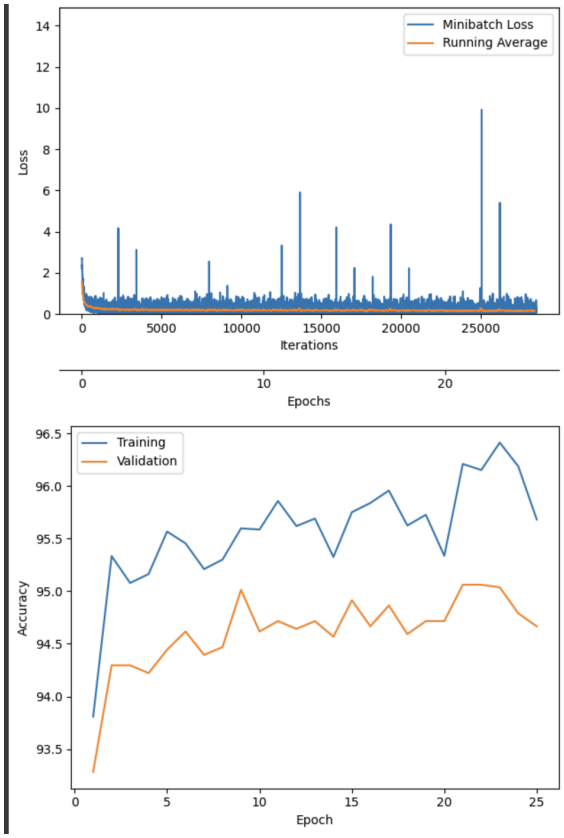


LRScheduler was used to improve training stability - say precise fine-tuning the learning rate while finding the minima, ReduceLROnPlateau was used because it allows LR to adapt based on model's performance. It showed a slight increase in accuracy.

Later used preactivation batchnorm - to stabilize and accelerate training - providing a regularizing effect. It improved the accuracy.



Later observed a slight overfitting in the train and validation curves, so added dropout to the layers after the activation with a lower dropout rate = 0.3 as the difference between the train and validation accuracy was not much.
This reduced the overfitting - and gave a better test accuracy.



5. Test accuracies :

Please Note : All test accuracies are included along with the hyperparameter tables.

For best models, test accuracies are as follows:

Models	**Test Accuracies**
CNN	**93.93**
RNN	**86.24**
Autoencoder	**94.84**